

Cortex Code - Product Requirements Document (PRD)

Version: 1.0.0

Status: Draft

Product Name: Cortex Code

Target Platforms:

- Visual Studio Code
 - JetBrains IDEs (Future)
 - Neovim (Future)
-

1. Vision

Cortex Code is an AI-powered coding assistant designed to provide an experience comparable to GitHub Copilot, Claude Code, and Cursor while remaining model-agnostic.

Unlike most coding assistants, Cortex Code should work with any OpenAI-compatible API.

Supported providers include:

- OpenAI
- Ollama
- NVIDIA NIM
- OpenRouter
- Together AI
- Groq
- LM Studio
- vLLM
- LocalAI
- Azure OpenAI
- Any OpenAI Specification compatible endpoint

The extension should support both cloud-hosted and local models.

2. Goals

- Fast AI coding assistance
- Agentic coding workflows
- Project-wide understanding
- Multi-file editing

- Inline code completion
 - Local-first compatibility
 - Secure tool execution
 - Extensible architecture
 - MCP support
 - RAG support
 - Provider-independent API
-

3. Non Goals

The extension will not:

- Require a proprietary backend
 - Lock users into one provider
 - Upload projects automatically
 - Execute dangerous commands without permission
-

4. Supported IDEs

Phase 1

- VS Code

Phase 2

- JetBrains IDEs

Phase 3

- Neovim
-

5. Core Features

5.1 AI Chat

Capabilities

- General chat
- Explain code
- Generate code
- Debugging
- Refactoring
- Architecture discussion

- Documentation generation
- Test generation
- Error explanation
- Performance optimization

Requirements

- Markdown rendering
 - Syntax highlighting
 - Streaming responses
 - Code block actions
 - Copy button
 - Insert into editor
 - Continue generation
 - Regenerate
 - Stop generation
-

5.2 Code Completion

Ghost text completion.

Requirements

- Single line completion
- Multi-line completion
- Function completion
- Import completion
- Class completion
- Comment completion

Features

- Tab accept
- Partial accept
- Next suggestion
- Previous suggestion
- Auto trigger
- Manual trigger

Performance

- Under 150 ms latency (target)
 - Cancellation support
 - Debouncing
-

5.3 Inline Edit

Select code

Invoke:

Ctrl + I

Examples

- Optimize
- Explain
- Convert
- Refactor
- Fix
- Simplify

Flow

1. User selects code
 2. Prompt appears
 3. LLM generates edit
 4. Diff shown
 5. User accepts/rejects
-

5.4 Agent Mode

The agent should complete complex programming tasks autonomously.

Example

Implement JWT authentication with refresh tokens.

Agent loop

1. Understand task
2. Search workspace
3. Read files
4. Create plan
5. Modify files
6. Run tests
7. Fix failures
8. Finish

Capabilities

- Read files
- Search workspace
- Edit files
- Create files
- Delete files
- Rename files
- Execute terminal commands

- Run tests
 - Read diagnostics
 - Observe outputs
 - Iterate until complete
-

5.5 Multi-file Editing

Examples

- Convert JavaScript to TypeScript
- Rename architecture
- Add logging
- Upgrade React
- Fix lint errors

Requirements

- Diff preview
 - Batch accept
 - Batch reject
 - Undo support
-

5.6 Workspace Understanding

Index

- Files
- Symbols
- Imports
- Classes
- Functions
- Variables
- Dependencies

Questions

- Where is authentication?
 - Where is login implemented?
 - Where is API client?
 - Show payment flow.
-

5.7 Terminal Integration

Supported

- npm

- yarn
- pnpm
- cargo
- python
- pip
- poetry
- docker
- git
- make

Permissions

Always ask before execution.

Show

- stdout
 - stderr
 - exit code
-

5.8 Git Integration

Capabilities

- Git status
 - Git diff
 - Git blame
 - Commit generation
 - Branch information
 - Review changes
 - Explain commits
 - PR review
-

5.9 Prompt Library

Built-in prompts

- Explain
- Optimize
- Review
- Fix
- Test
- Docs
- Refactor

User prompts

- Save

- Edit
 - Delete
 - Share
-

6. Context Collection

Automatically collect

- Current file
 - Open files
 - Selected text
 - Cursor position
 - Diagnostics
 - Git diff
 - Workspace structure
 - Recent edits
 - Terminal output
 - README
 - Package files
 - Lock files
 - Dependency graph
-

7. Tool System

File Tools

- Read File
- Write File
- Append File
- Delete File
- Rename File
- Move File
- Copy File

Directory Tools

- List Directory
- Create Directory
- Delete Directory

Search Tools

- Grep
- Ripgrep
- Regex Search
- Semantic Search

- Symbol Search

Git Tools

- Status
- Diff
- Commit
- Checkout
- Branch
- Log

Terminal Tools

- Execute
- Kill Process
- Stream Output

IDE Tools

- Diagnostics
 - Hover
 - References
 - Rename Symbol
 - Format Document
-

8. MCP Support

Support

- stdio
- HTTP
- SSE

Example MCP Servers

- Filesystem
- GitHub
- Browser
- PostgreSQL
- SQLite
- Docker
- Kubernetes
- Slack
- Notion

Users can register custom MCP servers.

9. Provider System

Every provider implements the same interface.

```
interface Provider {  
    chat()  
    streamChat()  
    completion()  
    embeddings()  
    listModels()  
    cancel()  
}
```

10. Supported Providers

Native Support

- OpenAI
- Ollama
- NVIDIA NIM
- OpenRouter
- Together AI
- Groq
- Azure OpenAI
- LM Studio
- vLLM
- LocalAI

Custom OpenAI Endpoint

Configuration

- Base URL
 - API Key
 - Model
 - Headers
 - Timeout
 - Temperature
 - Max Tokens
-

11. Model Support

Examples

- GPT-5
- GPT-4.1
- GPT-4o
- Claude 4
- DeepSeek
- Qwen
- Llama
- Gemma
- Mistral
- Phi

Users may manually specify any model.

12. RAG

Project indexing

Store

- Embeddings
- Symbols
- Metadata

Ignore

- node_modules
- dist
- build
- target
- .git
- coverage
- vendor
- out

Requirements

Incremental indexing

Automatic updates

Embedding cache

13. Memory

Workspace memory

Remember

- Coding style
- Architecture
- Framework preferences
- Ignore folders
- Naming conventions

Global memory

Optional

14. Slash Commands

```
/explain  
/fix  
/refactor  
/review  
/test  
/docs  
/search  
/git  
/terminal  
/new  
/optimize
```

15. Mentions

```
@workspace  
@file  
@folder  
@terminal  
@git  
@diff  
@clipboard  
@selection  
@docs  
@problems
```

16. Commands

Command Palette

- Cortex Chat
 - Cortex Agent
 - Explain
 - Fix
 - Refactor
 - Optimize
 - Generate Tests
 - Generate Docs
 - Review Code
 - Rename
 - Summarize
 - Commit Message
 - Toggle Completion
-

17. Settings

General

- Provider
- Base URL
- API Key
- Model

Generation

- Temperature
- Max Tokens
- Top P

Completion

- Enable
- Delay
- Trigger Mode

Agent

- Enable
- Auto Approvals
- Safe Mode

RAG

- Auto Index
- Embedding Model

UI

- Theme
 - Font Size
 - Streaming
-

18. Security

Never execute without permission

Protected operations

- Delete files
- Git push
- Git commit
- Install packages
- Shell execution
- Network requests

Permission Modes

- Ask Every Time
 - Allow This Session
 - Always Allow
-

19. Performance Requirements

Chat response

- First token under 2 seconds (network dependent)

Completion

- Under 150 ms target

Workspace indexing

- Background
- Incremental

Memory

- Lazy loading
- Cache management

Streaming

- Required

Cancellation

- Required
-

20. UI Components

Sidebar

- Chat
- History
- Prompts
- Agent Tasks

Editor

- Inline completion
- Code actions
- Hover actions

Status Bar

- Provider
- Model
- Token usage

Output Panel

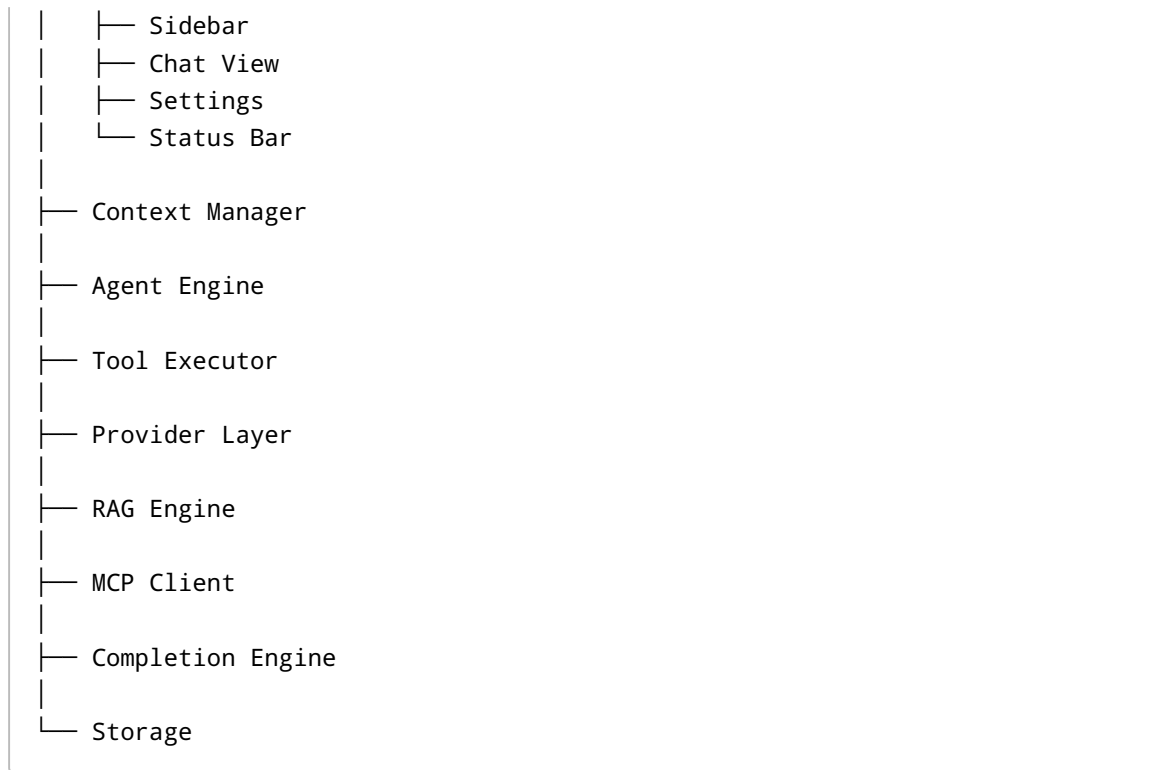
- Logs
- Agent execution
- Terminal

Settings UI

21. Architecture

VS Code Extension

└─ UI Layer



22. Suggested Tech Stack

Language

TypeScript

Extension

VS Code Extension API

UI

React

Vite

Monaco Editor

Parser

Tree-sitter

Search

Ripgrep

Storage

SQLite

sqlite-vec

Embeddings

OpenAI-compatible API

Communication

HTTP

Server-Sent Events (SSE)

WebSocket (optional)

23. Roadmap

Version 1.0

- Chat
- Completion
- Inline edits
- Provider support
- Streaming
- Git integration

Version 1.5

- Agent mode
- Multi-file edits
- Terminal
- RAG

Version 2.0

- MCP
- Memory
- Documentation search
- Image understanding

- Voice commands

Version 3.0

- Team collaboration
 - Cloud sync
 - Shared prompts
 - AI pull request reviews
 - Distributed agents
-

24. Success Metrics

- First response latency under 2 seconds
 - Completion acceptance rate above 35%
 - Crash-free sessions above 99%
 - Workspace indexing under 60 seconds for medium projects
 - Multi-provider compatibility across all supported OpenAI-compatible APIs
 - Zero destructive actions without explicit user approval
-

25. Future Ideas

- Browser automation
 - Mobile companion app
 - AI code review pipelines
 - Visual architecture diagrams
 - Local fine-tuned models
 - Plugin marketplace
 - Custom agent workflows
 - Multi-agent collaboration
 - Voice-driven development
 - Project health dashboards
-

End of Document